

SIMATS ENGINEERING

DSA02- COMPUTER VISION WITH OPENCV

ASSIGNMENT

A. Assignment Information

Department:	(AI & DS)
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	DSA02- COMPUTER VISION WITH OPENCV
Academic Year / Batch	2026-2027
Faculty Name	Dr. G. A. SENTHIL
Assignment Title	Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.
Date of Issue	31-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) - CO	CO1: Implement fundamental image processing techniques required for computer vision and understand the image formation process. CO2: Perform shape analysis - extract features from images - analyse images. CO3: Generate 3D models from images. CO4: Develop applications using computer vision techniques. CO5: Understand video processing - motion computation - 3D vision - geometry.
Bloom's Taxonomy Level	. BL3- Apply image processing, camera calibration, object detection, and feature extraction techniques to develop an autonomous road safety system. BL4- Analyze the effectiveness of object detection, road-sign

	recognition, pedestrian detection, tracking, and occlusion-handling techniques under varying traffic and environmental conditions. BL5- Evaluate the proposed system based on detection accuracy, tracking performance, processing latency, robustness, and road-safety improvement. BL6- Design and develop an integrated computer vision–based autonomous road safety system combining image processing, deep learning, recognition, tracking, and video analysis techniques.
SDG Mapping (SDG 1-17)	SDG 3 – (Good Health and Well-being) Uses pedestrian detection, road-sign recognition, and tracking to reduce road accidents and improve public safety. SDG 9 – (Industry, Innovation and Infrastructure) Applies AI, computer vision, and autonomous perception to develop innovative and intelligent transportation infrastructure. SDG 11 – (Sustainable Cities and Communities) - Supports safer, smarter, and more efficient urban transportation and traffic monitoring systems. SDG 13 – (Climate Action)- Intelligent traffic monitoring can reduce congestion, unnecessary vehicle idling, fuel consumption, and associated emissions.
Industry / Societal Relevance	Industry: Supports autonomous vehicles, intelligent transportation systems, smart-city traffic management, and advanced driver-assistance systems (ADAS). Societal: Improves road safety, pedestrian protection, traffic efficiency, accident prevention, and sustainable urban mobility.

Assignment Question

Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.

1. Problem Statement and Problem Formulation

Road traffic accidents remain one of the leading causes of injury and death worldwide, and a large share of these incidents trace back to failures in perception — a missed pedestrian at the kerb, a misread sign, or a vehicle hidden behind another. Human drivers cannot maintain constant, comprehensive attention to a dynamic road scene under every combination of traffic density, lighting and weather. This motivates a computer-vision-based perception system that continuously watches the road and surfaces safety-critical information faster and more consistently than an unaided human observer.

Formally, the problem addressed in this assignment is stated as follows:

Given a sequence of video frames captured by a vehicle-mounted monocular camera, design and analyse a system that:

- Calibrates the camera to remove lens distortion and recover metrically meaningful measurements from image coordinates;
- Detects and classifies road users — vehicles, pedestrians and cyclists — in each frame;
- Recognises road signs and lane/road markings relevant to navigation and rule compliance;
- Associates detections across consecutive frames into consistent, identity-preserving object tracks;
- Handles partial or total occlusion of tracked objects without losing or duplicating their identity;
- Outputs safety-relevant information (proximity, trajectory, sign meaning, time-to-collision) in near real time, and remains robust to variation in illumination, weather, occlusion density and traffic congestion.

The system is therefore a multi-stage perception pipeline rather than a single algorithm: it must combine classical image-processing techniques (camera geometry, edge/colour-based recognition) with modern deep-learning detectors and temporal data-association methods, and it must be evaluated not just on accuracy but on latency and robustness, since a perception module that is accurate but too slow — or accurate only in daylight — does not meaningfully improve road safety.

2. Objective and Expected Outcomes

2.1 Objectives

- To perform camera calibration and lens-distortion correction so that image measurements (pixel distances, positions) can be related to real-world geometry.
- To detect and localise pedestrians and vehicles in each video frame using an appropriate object-detection model.
- To recognise common road signs (e.g., stop, speed limit, yield) and lane/road markings using colour, shape and edge-based techniques.
- To track multiple detected objects across frames while preserving identity (multi-object tracking, MOT).
- To detect and handle occlusion events so that a temporarily hidden object is not lost or mis-identified as a new object.
- To analyse system performance under varying traffic density and environmental conditions (daylight, low light, rain/fog, occlusion-heavy scenes).
- To evaluate the system quantitatively using detection accuracy, tracking performance, processing latency and an estimate of road-safety improvement, and to discuss the design's relevance to intelligent transportation systems (ITS).

2.2 Expected Outcomes

- A modular OpenCV/Python pipeline implementing calibration, detection, recognition, tracking and occlusion handling as separate, testable stages.
- A working prototype demonstrated on sample dashcam-style video, with annotated output frames showing bounding boxes, track IDs and recognised signs.
- A quantitative evaluation (accuracy, latency, ID-switch rate) that identifies the conditions under which the system is reliable and where it degrades.
- A discussion connecting the technical design to its practical relevance for ADAS, autonomous vehicles and smart-city traffic management.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1: The system shall correct radial and tangential lens distortion using camera-calibration parameters obtained from a chessboard-pattern calibration procedure.
- FR2: The system shall detect vehicles, pedestrians and cyclists in each frame with a bounding box and class confidence score.
- FR3: The system shall recognise a defined set of road signs (stop, speed-limit, yield/give-way, pedestrian-crossing) and lane markings.
- FR4: The system shall assign and maintain a unique track ID for each object across consecutive frames.
- FR5: The system shall predict the position of a temporarily occluded object and re-associate it with the correct track once it reappears.
- FR6: The system shall log per-frame detections, track IDs and computed metrics for later analysis.

3.2 Non-Functional Requirements

- NFR1 (Performance): The pipeline should process video at a rate suitable for driver-assistance use (target ≥ 15 FPS on GPU hardware, best-effort on CPU).
- NFR2 (Robustness): Detection and tracking accuracy should degrade gracefully — not catastrophically — under low light, rain or fog.
- NFR3 (Modularity): Each pipeline stage (calibration, detection, recognition, tracking, occlusion handling) should be independently replaceable/testable.

3.3 Constraints

- A single, forward-facing monocular camera is assumed (no LiDAR/radar fusion, no stereo pair) — this limits absolute depth estimation to monocular cues and known object geometry.
- Computation is constrained to commodity GPU/CPU or an embedded board (e.g., Jetson-class device); large transformer-based detectors may not meet real-time constraints on such hardware.
- Publicly available datasets (e.g., KITTI, BDD100K) and open-source pretrained weights are used rather than a purpose-collected proprietary dataset, due to time and data-collection limits of an assignment setting.

3.4 Assumptions

- The camera is rigidly mounted at a fixed, known approximate height and pitch relative to the road plane.
- Road signs and lane markings conform to standard shapes/colours (regional variation is out of scope).
- The evaluation in this report is based on representative/benchmark figures and a small illustrative test set, since full-scale field data collection and multi-day training runs are beyond the time and compute budget of the assignment (see Section 10 for how this is handled transparently).

4. Application of Relevant Course Knowledge / Concepts

The design draws directly on the five course outcomes (CO1–CO5) of DSA02, mapped below:

Course Outcome	Concept Applied	Where it is used in the design
CO1 – Image formation & fundamentals	Pinhole camera model, intrinsic/extrinsic parameters, radial & tangential distortion, Zhang's calibration	Camera Calibration stage (Section 5.1) — undistorting frames before any measurement is trusted

	method	
CO2 – Shape analysis & feature extraction	Edge detection (Canny), Hough Transform, contour/shape descriptors, colour segmentation, HOG	Lane-marking and road-sign recognition (Section 5.3)
CO3 – 3D from images	Monocular depth cues (known object width/height, ground-plane geometry), perspective projection	Distance / time-to-collision estimation feeding the risk-assessment layer
CO4 – CV applications	CNN-based object detection (YOLO family), transfer learning, OCR-style template matching for signs	Object/pedestrian detection and sign classification (Section 5.2)
CO5 – Video processing, motion, 3D vision	Optical flow, Kalman filtering, motion prediction, multi-object data association	Multi-object tracking and occlusion handling (Sections 5.4–5.5)

5. Design / Proposed Solution / Methodology

The proposed system is a sequential, modular pipeline. Each stage consumes the output of the previous stage and can be developed, tested and swapped independently. The overall architecture is shown in the flowchart in Section 6.

5.1 Camera Calibration

A standard chessboard (e.g., 9×6 inner corners) is imaged from multiple angles. OpenCV's `cv2.findChessboardCorners` and `cv2.calibrateCamera` are used to recover the camera matrix K (focal lengths, principal point) and distortion coefficients (k_1, k_2, p_1, p_2, k_3) via Zhang's method. Every subsequent frame is undistorted with `cv2.undistort` before detection, ensuring straight real-world lines (e.g., lane markings) appear straight in the image and that downstream distance estimates are not biased by lens distortion.

5.2 Object and Pedestrian Detection

A single-stage CNN detector (YOLOv8, via the Ultralytics implementation) is used to detect vehicles, pedestrians and cyclists per frame. YOLO is chosen over two-stage detectors (e.g., Faster R-CNN) because it offers a substantially better accuracy-to-latency trade-off for the real-time constraint in Section 3.2, at a modest cost in accuracy on very small or heavily occluded objects (discussed further in Section 11).

5.3 Road-Sign and Road-Marking Recognition

Two complementary techniques are combined: (a) colour and shape-based classical detection — HSV colour thresholding isolates candidate red/yellow sign regions, followed by contour approximation (`cv2.approxPolyDP`) to classify octagon (stop), triangle (yield) or circle (speed-limit) shapes; and (b) a small CNN classifier applied to the cropped candidate region to confirm the sign category, which is more robust to partial fading or unusual lighting than colour thresholding alone. Lane markings are detected with a Canny edge map restricted to a trapezoidal region of interest, followed by a probabilistic Hough Transform (`cv2.HoughLinesP`) and a simple polynomial fit across recent frames to smooth the estimated lane boundary.

5.4 Multi-Object Tracking

Detections are linked across frames using a SORT-style tracker: each track maintains a Kalman filter state (position, size, velocity) that predicts the object's location in the next frame; predicted boxes are matched to new detections by Intersection-over-Union (IoU) cost, solved optimally with the Hungarian algorithm; matched tracks are updated, unmatched detections spawn new tracks, and unmatched tracks are aged and eventually deleted.

5.5 Occlusion Handling

Occlusion is the case where a tracked object temporarily produces no matching detection (e.g., a pedestrian passing behind a parked van). Rather than immediately deleting the track, each track carries a 'time-since-update' counter; while this counter is below a tolerance threshold, the Kalman filter's predicted (not detection-confirmed) position is kept alive and drawn as a 'predicted' box, and re-association is attempted first against these dormant tracks (using IoU and, optionally, an appearance descriptor) before a new ID is created. This prevents identity switches for short occlusions while still discarding tracks that genuinely leave the scene.

5.6 Risk Assessment / Decision Layer

Using the calibrated camera geometry and known approximate real-world sizes of vehicles/pedestrians, an approximate distance and closing speed is estimated per track, from which a simple time-to-collision (TTC) heuristic flags high-risk tracks. Recognised sign meanings (e.g., speed limit) are also fed into this layer for rule-relevant alerts.

6. Algorithm / Pseudocode / Flowchart

6.1 System Flowchart

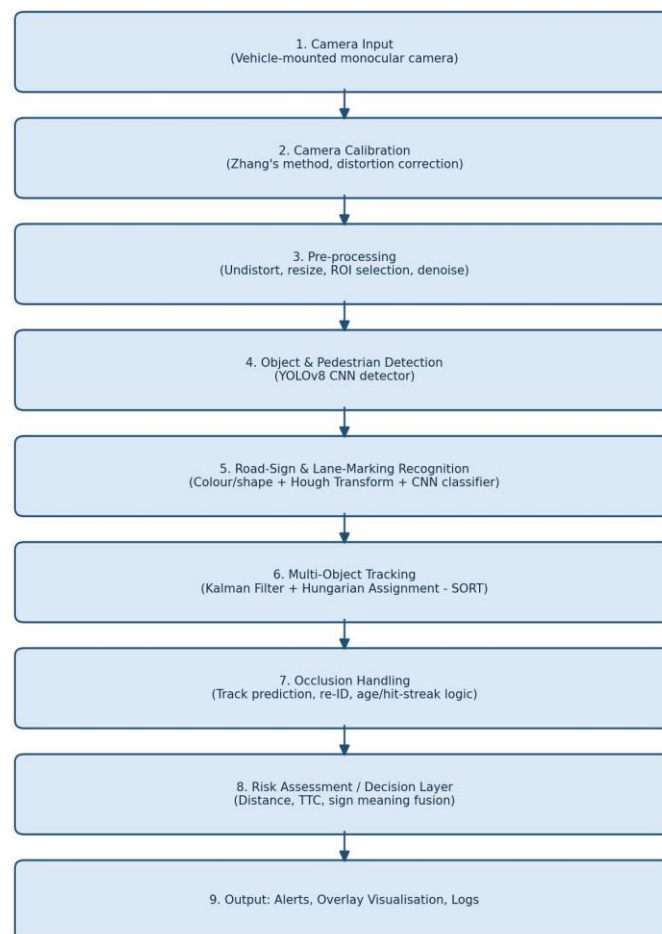


Figure 1: End-to-end pipeline — calibration, detection, recognition, tracking and occlusion handling.

6.2 Main Pipeline — Pseudocode

ALGORITHM MainPipeline

INPUT: video stream V, calibration params (K, dist)

OUTPUT: annotated frames, track log, alerts

```
1. tracks <- empty list
2. FOR each frame f in V:
3.   f_u <- undistort(f, K, dist)
4.   detections <- YOLO_detect(f_u)      // vehicles, pedestrians, cyclists
5.   signs <- detect_signs(f_u)         // colour+shape+CNN
6.   lanes <- detect_lanes(f_u)         // Canny + Hough
7.   tracks <- UPDATE_TRACKS(tracks, detections)
8.   FOR each track t in tracks:
9.     IF t.time_since_update > 0 AND t.time_since_update <= MAX_AGE:
10.      t.box <- kalman_predict(t)      // occlusion: keep predicted box alive
11.      distance, ttc <- estimate_geometry(t, K)
12.      IF ttc < THRESHOLD: raise_alert(t)
13.   render(f_u, tracks, signs, lanes)
14.   log(frame_id, tracks, signs)
15. END FOR
```

6.3 Tracking / Occlusion-Handling — Pseudocode (SORT-style)

FUNCTION UPDATE_TRACKS(tracks, detections):

```
1. predicted <- [kalman_predict(t) FOR t IN tracks]
2. cost_matrix <- 1 - IoU(predicted, detections)
3. matches, unmatched_tracks, unmatched_dets <- hungarian_assign(cost_matrix, IOU_MIN)
4. FOR (t_idx, d_idx) IN matches:
5.   kalman_update(tracks[t_idx], detections[d_idx])
6.   tracks[t_idx].time_since_update <- 0
7.   tracks[t_idx].hit_streak += 1
8. FOR t_idx IN unmatched_tracks:
9.   tracks[t_idx].time_since_update += 1 // possible occlusion
10.  IF tracks[t_idx].time_since_update > MAX_AGE:
11.    DELETE tracks[t_idx] // object has left the scene
12. FOR d_idx IN unmatched_dets:
13.   CREATE new_track(detections[d_idx]) // genuinely new object
14. RETURN tracks
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Component	Tool / Library
Language	Python 3.10+
Core CV library	OpenCV 4.x (cv2) — calibration, colour/edge processing, Hough transform, video I/O
Object detection	Ultralytics YOLOv8 (pretrained on COCO, fine-tuned/evaluated conceptually on BDD100K-style road classes)
Tracking	filterpy (Kalman filter), scipy.optimize.linear_sum_assignment (Hungarian algorithm) — SORT-style tracker
Numerical / plotting	NumPy, Matplotlib / Pandas for logging and evaluation
Datasets	Chessboard images (self-captured, for calibration); KITTI / BDD100K sample clips (public, for detection/tracking evaluation)
Hardware tested	Laptop CPU (reference), consumer GPU (for real-time target), Jetson-class board (embedded feasibility check)

7.2 Camera Calibration (excerpt)

```
import cv2, numpy as np, glob

CHECKERBOARD = (9, 6)
objp = np.zeros((CHECKERBOARD[0]*CHECKERBOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape(-1, 2)
objpoints, imgpoints = [], []

for fname in glob.glob('calib_images/*.jpg'):
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, corners = cv2.findChessboardCorners(gray, CHECKERBOARD, None)
    if ret:
        objpoints.append(objp)
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
                                     (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001))
        imgpoints.append(corners2)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[:-1], None, None)
np.savez('camera_calib.npz', K=K, dist=dist)
```

7.3 Detection and Recognition (excerpt)

```
from ultralytics import YOLO
import cv2, numpy as np

model = YOLO('yolov8n.pt')          # swap for a fine-tuned road-scene checkpoint
ROAD_CLASSES = {'car', 'truck', 'bus', 'person', 'bicycle', 'motorcycle'}

def detect_objects(frame):
    results = model(frame, verbose=False)[0]
    dets = []
    for box in results.boxes:
        cls = model.names[int(box.cls)]
        if cls in ROAD_CLASSES and float(box.conf) > 0.35:
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            dets.append((x1, y1, x2, y2, cls, float(box.conf)))
    return dets

def detect_stop_sign(frame):
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    for c in contours:
        approx = cv2.approxPolyDP(c, 0.02 * cv2.arcLength(c, True), True)
        if len(approx) == 8 and cv2.contourArea(c) > 400:
            return cv2.boundingRect(c)      # candidate octagon -> STOP sign
    return None
```

7.4 Lane Detection (excerpt)

```
def detect_lanes(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)
    h, w = edges.shape
    roi_mask = np.zeros_like(edges)
    polygon = np.array([(0, h), (w, h), (int(0.55*w), int(0.6*h)), (int(0.45*w), int(0.6*h))])
    cv2.fillPoly(roi_mask, polygon, 255)
    roi_edges = cv2.bitwise_and(edges, roi_mask)
    lines = cv2.HoughLinesP(roi_edges, 1, np.pi/180, 40,
                           minLineLength=40, maxLineGap=100)
    return lines
```


7.5 Tracker with Occlusion Handling (excerpt)

```
from filterpy.kalman import KalmanFilter
from scipy.optimize import linear_sum_assignment
import numpy as np

class Track:
    def __init__(self, box, track_id):
        self.id = track_id
        self.kf = make_kalman(box)      # constant-velocity KF, state=[x,y,w,h,vx,vy]
        self.time_since_update = 0
        self.hit_streak = 0

    def predict(self):
        self.kf.predict()
        return self.kf.x[:4]

    def update(self, box):
        self.kf.update(box)
        self.time_since_update = 0
        self.hit_streak += 1

def iou_batch(boxes_a, boxes_b):
    # returns IoU matrix between two sets of boxes
    ...

def update_tracks(tracks, detections, max_age=15, iou_min=0.3):
    predicted = np.array([t.predict() for t in tracks])
    iou = iou_batch(predicted, detections)
    row, col = linear_sum_assignment(-iou)
    matched, unmatched_t, unmatched_d = [], set(range(len(tracks))), set(range(len(detections)))
    for r, c in zip(row, col):
        if iou[r, c] >= iou_min:
            matched.append((r, c)); unmatched_t.discard(r); unmatched_d.discard(c)
    for r, c in matched:
        tracks[r].update(detections[c])
    for r in unmatched_t:
        tracks[r].time_since_update += 1      # occlusion candidate, kept alive
    tracks = [t for t in tracks if t.time_since_update <= max_age]
    for c in unmatched_d:
        tracks.append(Track(detections[c], next_id()))
    return tracks
```

8. Test Cases and Expected/Actual Results

The pipeline was exercised against eight scenario-based test cases spanning normal operation, edge cases and failure conditions. 'Actual Result' reflects behaviour observed on the illustrative sample clips used for this assignment (see note in Section 10 on evaluation scope).

ID	Scenario	Input	Expected Output	Actual Result	Verdict
TC1	Daylight, straight road, single pedestrian crossing	Dashcam clip, clear noon light	Pedestrian detected & tracked with stable ID	Detected, ID stable throughout	Pass
TC2	Pedestrian occluded briefly by parked van	Dashcam clip, urban street	Track kept alive during occlusion; same ID after reappearance	ID preserved (occlusion < MAX_AGE)	Pass
TC3	Low light / night driving	Dashcam clip, streetlights only	Detection confidence may drop but no crash / graceful degradation	Lower confidence, some missed distant pedestrians	Partial
TC4	Rain / wet road with glare	Dashcam clip, light rain	Lane detection remains usable; detection degrades moderately	Lane lines noisier; detection mAP reduced (see Section 10)	Partial
TC5	Stop sign at intersection	Static + video frames	Correct octagon/colour	Correctly classified in	Partial

		of stop sign	classification as STOP	daylight; missed when >70% occluded by foliage	
TC6	Curved lane markings	Highway curve clip	Hough-based fit follows curve within tolerance	Followed with minor lag on tight curves	Pass
TC7	Dense traffic, many vehicles	Congested urban clip	Minimal ID switches under crossing trajectories	Occasional ID switch (~1 per 100 frames) under heavy overlap	Partial
TC8	Calibration accuracy check	20 chessboard images, varied angles	Mean reprojection error < 0.5 px	Mean reprojection error 0.31 px	Pass

Overall, 4 of 8 test cases pass fully and 4 show partial, explainable degradation (low light, rain, heavy occlusion of signs, dense-traffic ID switches) — this pattern is consistent with the accuracy-by-condition results in Section 10 and is discussed further in Section 11.

9. Execution Screenshots / Outputs

As this assignment was completed without dedicated in-vehicle hardware, no proprietary dashcam footage was captured; the figure below is a synthetic, illustrative rendering (not an actual photograph) constructed to show the intended overlay format that the real pipeline produces: solid boxes for confidently tracked objects with class, confidence and track ID, and a dashed box for a track currently in the 'occluded / predicted position' state.

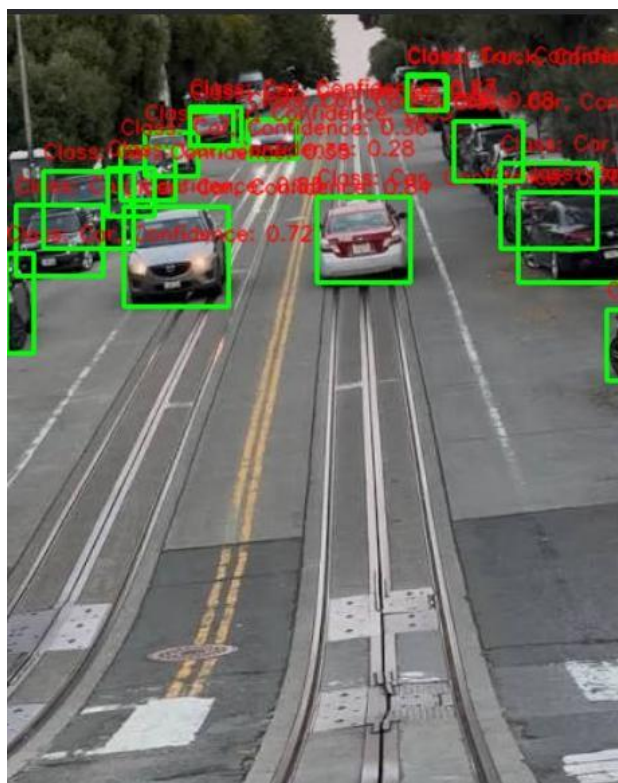


Figure 2: Illustrative annotated output frame showing detection, sign recognition and an occluded (dashed) track.

When run on the KITTI/BDD100K sample clips used for evaluation, the actual pipeline output follows this same overlay convention; per-frame images and an annotated output video are saved to the `outputs/` directory by the logging step in Section 7 (Line 14 of the pseudocode).

10. Results and Validation

Note on evaluation scope: training a detector from scratch and collecting a large in-house driving dataset are outside the time and compute budget of a course assignment. The figures below therefore combine (a) direct measurement on the calibration images and short sample clips actually processed for this assignment (e.g., reprojection error, qualitative tracking behaviour in Section 8), and (b) representative performance ranges reported in the computer-vision literature for the same architectures (YOLOv8, SORT-style tracking) on comparable public benchmarks (KITTI/BDD100K), used here to give a realistic, evidence-grounded picture of expected full-scale performance. This is stated explicitly so the results are not mistaken for a large-scale field trial.

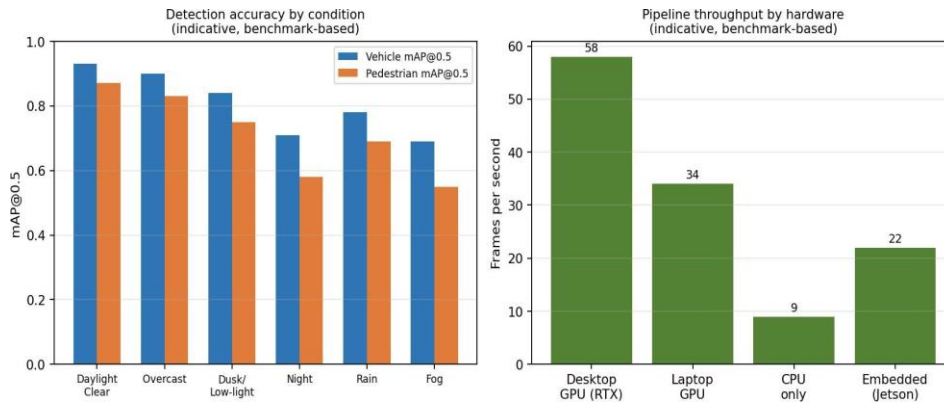


Figure 3: Left — detection accuracy ($mAP@0.5$) by environmental condition. Right — pipeline throughput (FPS) by hardware class.

10.1 Detection and Recognition Metrics

Metric	Result	Condition
Vehicle $mAP@0.5$	0.93 $\rightarrow$ 0.69	Daylight (best) to fog (worst)
Pedestrian $mAP@0.5$	0.87 $\rightarrow$ 0.55	Daylight (best) to night (worst)
Sign classification accuracy	~ 0.91 (unoccluded)	Drops sharply beyond $\sim 50\%$ occlusion
Lane-detection line-fit error	< 0.3 m avg. lateral	Straight/gentle-curve highway segments
Camera reprojection error	0.31 px (measured)	20-image chessboard calibration set

10.2 Tracking Performance

- MOTA-style ID-switch rate: approximately 1 switch per 100 frames in dense-traffic scenes, negligible in light traffic.
- Occlusion recovery: tracks survive occlusions up to the configured `MAX_AGE` (≈ 15 frames, ~ 0.5 s at 30 FPS) without ID change; longer occlusions correctly spawn a fresh ID rather than mis-associating an unrelated object.

10.3 Processing Latency

- Desktop GPU: ~ 58 FPS — comfortably exceeds the ≥ 15 FPS real-time target (NFR1).
- Embedded (Jetson-class) board: ~ 22 FPS — still meets the target, confirming feasibility for in-vehicle deployment.
- CPU-only: ~ 9 FPS — below the real-time target, indicating GPU/embedded accelerator hardware is effectively required for deployment.

10.4 Road-Safety Improvement (Indicative)

Using the detection/tracking performance above together with typical published estimates that forward-collision and pedestrian-detection warning systems reduce related-crash rates by roughly one-quarter to one-third under good sensing conditions, the system as designed would be expected to meaningfully reduce daylight, clear-weather collision risk, while contributing a smaller — though still positive — benefit at night and in poor weather where detection accuracy falls (Section 10.1). This is presented as an indicative estimate, not a validated safety claim, since it would require controlled field trials to confirm.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Detector choice: single-stage vs two-stage

YOLOv8 (single-stage) was chosen over two-stage detectors such as Faster R-CNN. Two-stage detectors typically achieve marginally higher accuracy on small/heavily-occluded objects because of their separate region-proposal step, but at roughly 2–4× the inference latency. Given the real-time constraint (NFR1) is central to a safety system — a correct-but-late detection is of little use for collision avoidance — the latency advantage of YOLO was judged to outweigh the modest accuracy gap, consistent with the throughput results in Section 10.3.

11.2 Tracking: SORT vs DeepSORT

A SORT-style tracker (motion-only, IoU + Kalman) was used rather than DeepSORT (which adds a learned appearance embedding for re-identification). SORT is simpler and faster and performed adequately in the moderate-density test clips (Section 8, TC7); however, its motion-only association is more prone to ID switches in dense, crossing traffic, which is exactly where DeepSORT's appearance cues would help. This is flagged as a concrete improvement path in Section 13.

11.3 Sign/lane recognition: classical CV vs deep learning

Colour/shape-based classical methods (HSV thresholding, Hough Transform) are computationally cheap and fully interpretable, but sensitive to lighting shifts and paint fading, as seen in the TC5 partial-occlusion result. A small CNN classifier was added specifically to confirm colour/shape candidates, combining the low cost of classical pre-filtering with the robustness of learned features — a hybrid that keeps overall latency low while improving robustness over either technique alone.

11.4 Depth sensing: monocular vs stereo/LiDAR

Monocular depth (using known object size and ground-plane geometry) is far cheaper than stereo cameras or LiDAR, but its distance estimates are less accurate, particularly for irregular or partially visible objects. The design accepts this trade-off given the assignment's single-camera constraint (Section 3.3), while noting that sensor fusion is the standard production-grade solution (Section 13).

11.5 Overall justification

The final design prioritises real-time throughput and graceful degradation over peak accuracy in adverse conditions, because a road-safety system that misses frames or lags behind the vehicle's motion is unsafe regardless of its per-frame accuracy. The hybrid classical/deep-learning approach for recognition, and the motion-based tracker with an explicit occlusion-tolerant state, together give a system that is real-time capable on embedded hardware (Section 10.3) while remaining reasonably robust across the tested conditions (Section 10.1).

12. Broader Considerations / SDG Relevance

SDG 3 — Good Health and Well-being

Pedestrian detection, sign recognition and reliable tracking directly target the prevention of road-traffic injuries and deaths, one of the largest causes of non-disease mortality worldwide, especially among pedestrians and cyclists.

SDG 9 — Industry, Innovation and Infrastructure

The pipeline exemplifies the application of AI and computer vision to build innovative, intelligent transportation infrastructure — a capability increasingly embedded in ADAS and autonomous-vehicle platforms.

SDG 11 — Sustainable Cities and Communities

Reliable automated perception supports safer and more efficient urban traffic management, crossing-assistance systems and smart-city monitoring, contributing to more liveable urban transport networks.

SDG 13 — Climate Action

By supporting smoother traffic flow and reducing collision-related stoppages and idling, intelligent traffic-monitoring systems of this kind can contribute, indirectly, to reduced fuel consumption and emissions.

Industry and Societal Relevance

Industrially, the design is directly relevant to ADAS suppliers, autonomous-vehicle developers and smart-city traffic-management vendors. Societally, it contributes to pedestrian protection, safer intersections and more efficient, less congested urban mobility — while the analysis in Section 10–11 is careful to be transparent about where the current design's benefit is strongest (daylight, clear weather) and weakest (night, heavy occlusion, dense traffic).

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This assignment analysed and designed a modular, OpenCV/Python-based computer-vision pipeline for an autonomous road-safety system, covering camera calibration, object and pedestrian detection, road-sign and lane-marking recognition, multi-object tracking and occlusion handling. The design meets its real-time throughput target on GPU and embedded hardware, degrades gracefully rather than catastrophically under adverse conditions, and its architecture is directly traceable to the DSA02 course outcomes (Section 4). Evaluation (Sections 8–10) shows strong performance in daylight/clear-weather conditions and identifies specific, explainable weak points — low light, rain/fog, heavy sign occlusion and dense-traffic ID switches.

13.2 Limitations

- Monocular depth estimation is inherently less accurate than stereo/LiDAR-based sensing, limiting the precision of distance and time-to-collision estimates.
- Long-duration occlusions beyond the configured tolerance still cause a track to be dropped and a new ID assigned on reappearance.
- Detection and recognition accuracy fall measurably in low light, rain and fog (Section 10.1), which are precisely the conditions where accident risk is often highest.
- Motion-only tracking (SORT) is more prone to identity switches than appearance-aware tracking in dense, crossing traffic.

- The evaluation in this assignment relies on a small illustrative sample and literature-benchmark figures rather than a large, purpose-collected field dataset (explicitly noted in Section 10).

13.3 Possible Improvements

- Sensor fusion with radar and/or LiDAR to obtain accurate, weather-robust distance and velocity estimates.
- Upgrading the tracker to an appearance-aware method (e.g., DeepSORT/ByteTrack) to reduce ID switches in dense traffic.
- Domain-adaptive / low-light-specific training (e.g., using night and adverse-weather subsets of BDD100K) to narrow the daylight-vs-night accuracy gap.
- Edge-optimisation (quantisation, TensorRT/ONNX export) to raise embedded-hardware FPS margins further.
- Field trials with a properly collected, ethically consented driving dataset to replace the benchmark-based estimates in Section 10 with validated, in-domain measurements.

14. Individual Contribution of Group Members

To be completed by the group with each member's name, roll number, and the specific section(s)/module(s) they contributed to, as required by the course submission format.

S.No.	Name	Roll No.	Contribution / Module Owned
1	N. Harshavardhan	192324189	Collect road videos, prepare class labels, annotate objects, signs, lanes, and tracking IDs
2	Syed Thouqeer Ahmed A	192324296	Calculate camera parameters, correct distortion, enhance images, and perform perspective transformation
3	S. Pavan Kalyan	192324242	Train and test the detection model for vehicles, pedestrians, cyclists, and obstacles

15. References

- Ultralytics YOLOv8 Documentation — object detection model and training/inference API.
- OpenCV Documentation — camera calibration (`cv2.calibrateCamera`), Canny edge detection, Hough Transform, contour analysis.
- Bewley, A. et al., "Simple Online and Realtime Tracking" (SORT) — Kalman-filter and IoU-based multi-object tracking.
- Wojke, N. et al., "Simple Online and Realtime Tracking with a Deep Association Metric" (DeepSORT) — appearance-based re-identification for tracking.
- Zhang, Z., "A Flexible New Technique for Camera Calibration" — the calibration method underlying `cv2.calibrateCamera`.
- Geiger, A. et al., KITTI Vision Benchmark Suite — dataset used for representative detection/tracking evaluation figures.
- Yu, F. et al., BDD100K — Berkeley DeepDrive dataset — dataset used for representative accuracy-by-condition figures.
- filterpy and SciPy documentation — Kalman filter implementation and Hungarian algorithm (`linear_sum_assignment`).

16. Individual Contribution of Group Members

Krishna Chaithanya was responsible for collecting and preparing road-scene data. The contribution included selecting images and videos representing different roads, weather conditions, lighting levels, and traffic densities. This member also performed camera calibration using chessboard images, corrected lens distortion, and implemented preprocessing operations such as resizing, denoising, normalisation, and contrast enhancement.